

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

Кафедра компьютерных технологий и программной инженерии

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

М. Д. Поляк

инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики производственная

тип практики по получению профессиональных умений и опыта профессиональной деятельности

на тему индивидуального задания Модуль проверки отчетов в формате OpenOffice

выполнен Чжан Чаоюй

фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки

09.03.04

код

Программная инженерия

наименование направления

направленности

наименование направления

02

код

Проектирование программных систем

наименование направленности

наименование направленности

Обучающийся группы № 4233К

номер

подпись, дата

Чжан Чаоюй

инициалы, фамилия

Санкт–Петербург 2025

Содержание

Задача.....	3
Описание входных и выходных	3
Требования к работе модуля	5
Порядок выполнения работы	6
Демонстрация работоспособности программы	7
Выводы	10
Листинг кода	11

Задача

Реализовать модуль для проверки отчетов студентов, выполненных в формате OpenOffice (odt).

Описание входных и выходных

Входные данные:

- файл с отчетом в виде потока байт;
- словарь с информацией о студенте;
- словарь с информацией об отчете.

Выходные данные:

Список строк с описанием ошибок в отчете, по одной строке на каждую ошибку. Пример:

```
[  
    "Неправильное название предмета",  
    "Неверное ФИО студента",  
    "Неверная дата"  
]
```

Информация о студенте:

```
{  
    "name": "Иван",  
    "surname": "Иванов",  
    "patronymic": "Иванович",  
    "group": "4931"  
}
```

Поле *patronymic* может быть пустым (*None* в Python) или ключ может отсутствовать вовсе. Поле *group* может содержать буквы, поэтому тип — строка.

Информация об отчете:

```
{
  "subject_name": "Операционные системы",
  "task_name": "ЛР1. Знакомство с командным интерпретатором bash",
  "task_type": "Лабораторная работа",
  "teacher": {
    "name": "Юлия",
    "surname": "Антохина",
    "patronymic": "Анатольевна",
    "status": "Ректор, д.т.н., проф."
  },
  "report_structure": [
    "Цель", "Задание", "Результат выполнения", "Выводы"
  ],
  "uploaded_at": "2022-06-01T00:00:00Z"
}
```

Поле *uploaded_at* содержит дату загрузки студентом отчета в личный кабинет в формате ISO 8601 в часовом поясе UTC.

Требования к работе модуля

Для решения поставленной задачи необходимо написать функцию на Python, принимающую на вход все необходимые данные. Файл с кодом должен позволять как использование написанной функции в другом модуле с помощью команды *import*, так и самостоятельное выполнение кода как отдельного приложения с помощью команды *python <имя_файла>.py <аргументы>*.

В отчете необходимо проверять:

- титульный лист на соответствие заданным во входных данных параметрам:
 - тип задания (лабораторная работа, курсовой проект и т.п.),
 - ФИО студента,
 - группа студента,
 - название предмета,
 - название задания (лабораторной работы),
 - ФИО преподавателя,
 - должность преподавателя,
 - год выполнения отчета внизу титульного листа (проверять по дате загрузки отчета)
- содержимое отчета на наличие обязательных разделов, если таковые были указаны.

Программа должна поддерживать произвольный шаблон титульного листа, как скачанный с сайта ГУАП, так и упрощенный, а также любые их вариации, в т.ч. с любым оформлением (размер шрифта; выделение слов жирным, курсивом и т.п.; использование невидимых таблиц для форматирования текста, использование разрывов страниц и пр.).

Порядок выполнения работы

1. Провести анализ существующих библиотек языка Python для парсинга файлов заданного формата. Составить сравнительную таблицу, указав, в том числе: дату выхода последней версии библиотеки; лицензию, по которой распространяется библиотека; наличие внешних зависимостей, которые необходимо устанавливать отдельно; количество звезд на гитхабе, количество форков и открытых Issues; наличие подробной документации; и др.
2. Проанализировать функциональные возможности найденных библиотек для решения поставленной задачи.
3. Согласовать окончательный выбор библиотеки для парсинга файлов.
4. Реализовать поставленную задачу.
5. Подготовить тестовые данные (файлы с отчетами в нужном формате) для проверки корректности работы программы. На каждый проверяемый в отчете элемент должен быть как минимум один отдельный тест, в котором проверяется, что в случае некорректного заполнения соответствующей части отчета, написанная программа генерирует все необходимые сообщения о содержащейся в отчете ошибке. Предусмотреть тесты с различными вариантами оформления отчета (в т.ч. титульного листа). Также тестовые данные должны включать несколько правильно выполненных отчетов, а также неправильно выполненные отчеты, содержащие сразу несколько ошибок.
6. Подготовить тесты с использованием библиотеки *pytest*.
7. Написать CI/CD процесс с использованием GitHub Actions для автоматического запуска тестов при изменении файла (-ов) в репозитории, в которых реализован функционал данной задачи.

Демонстрация работоспособности программы

```
{  
    "name": "Владимир",  
    "surname": "Иванов",  
    "patronymic": "Петрович",  
    "group": "4133К"  
}
```

Рисунок 1 – Входные данные студента

```
{  
    "subject_name": "Операционные системы",  
    "task_name": "ЛР1. Знакомство с командным интерпретатором bash",  
    "task_type": "Лабораторная работа",  
    "teacher": {  
        "name": "Марк",  
        "surname": "Поляк",  
        "patronymic": "Дмитриевич",  
        "status": "Старший преподаватель"  
    },  
    "report_structure": [  
        "Цель", "Задание", "Результат выполнения", "Выводы"  
    ],  
    "uploaded_at": "2024-06-01T00:00:00Z"  
}
```

Рисунок 2 – Входные данные отчета

ГУАП КАФЕДРА № 43			
ОТЧЕТ ЗАЩИЩЕН С ОЦЕНКОЙ ПРЕПОДАВАТЕЛЬ			
Старший преподаватель			
должность, уч. степень, звание	подпись, дата		М. Д. Поляк инициалы, фамилия
ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1 ЗНАКОМСТВО С КОМАНДНЫМ ИНТЕРПРЕТАТОРОМ BASH по курсу: ОПЕРАЦИОННЫЕ СИСТЕМЫ			
РАБОТУ ВЫПОЛНИЛ			
СТУДЕНТ гр. №	4133К		
		подпись, дата	В. П. Иванов инициалы, фамилия

Рисунок 3 – Отчет

Цель
Задание
Результат выполнения
Выводы

Рисунок 4 – Отчет

```
Run  check_report_module x
D:\pythonProject2\.venv\Scripts\python.exe D:\pythonProject2\check_report\check_report_module.py
Должность преподавателя: Старший преподаватель
ФИО преподавателя: М.Д.Поляк
Тип отчета: Лабораторная работа
Название работы: ЛР1. Знакомство с командным интерпретатором bash
Название предмета: Операционные системы
Номер группы: 4133К
ФИО студента: В.П.Иванов
Структура отчета: Цель, Задание, Результат выполнения, Выводы
Дата загрузки: 2024-06-01

Process finished with exit code 0
```

Рисунок 5 – Результат выполнения программы

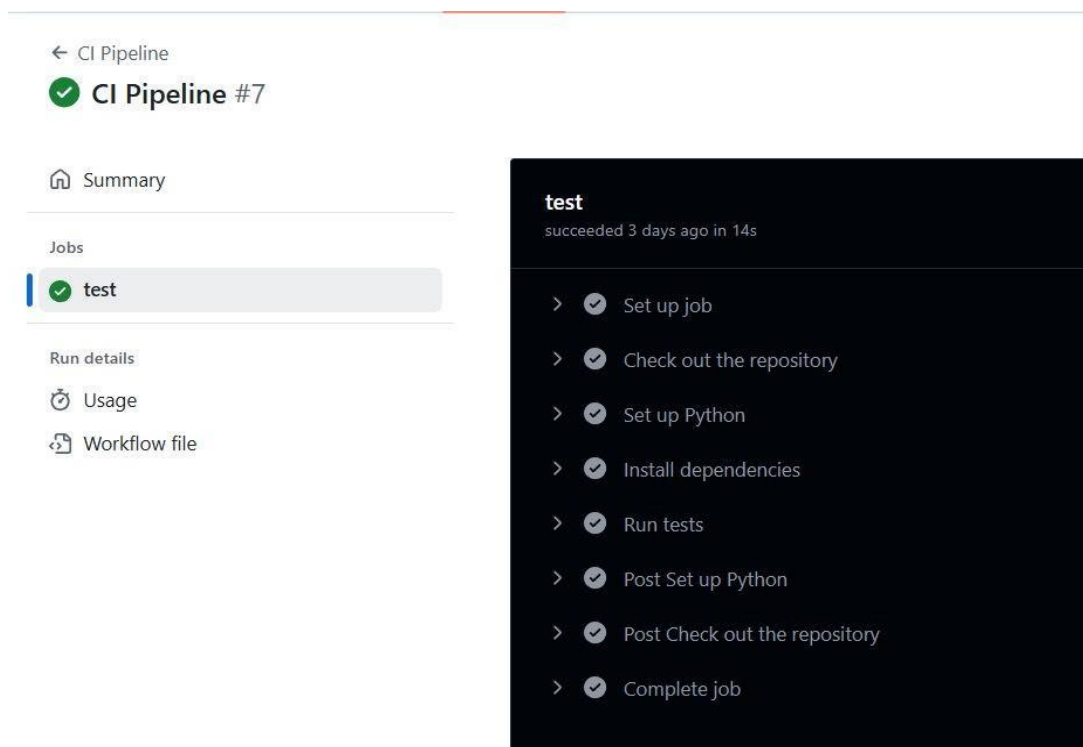


Рисунок 6 – Тесты GitHub Actions

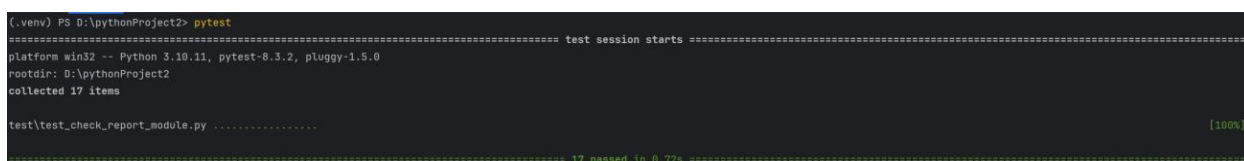


Рисунок 7 – Тесты pytest

Выводы

При выполнении практики был реализован модуль проверки отчетов в формате OpenOffice.

Листинг кода

MAIN.YML

```
name: CI Pipeline

on:
  push:
    paths:
      - 'check_report/**'
    branches:
      - main
  pull_request:
    paths:
      - 'check_report/**'
    branches:
      - main
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Check out the repository
        uses: actions/checkout@v2

      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest
          pip install odfp

      - name: Run tests
        run: pytest test/test_check_report/module.py
```

CHECK_REPORT_MODULE.PY

```
import io
import os
import json
import argparse

from datetime import datetime

from odf import opendocument
from odf.element import Element

STUDENT_JSON = {
    "name": "str",
    "surname": "str",
    "patronymic": "str | None",
    "group": "str"
}

REPORT_JSON = {
    "subject_name": "str",
    "task_name": "str",
    "task_type": "str",
    "teacher": {
        "name": "str",
        "surname": "str",
        "patronymic": "str | None",
        "status": "str"
    },
    "report_structure": [
        "str", "str", "str", "str"
    ],
    "uploaded_at": "str (ISO 8601 UTC)"
}

def remove_file_if_exists(file_path: str) -> None:
    if os.path.exists(file_path):
        os.remove(file_path)

def write_document_struct(element: Element, level=0) -> None:
    with open(file='../document_structure.txt', mode='a',
              encoding='utf-8') as file:
        file.write(' ' * level + f"{element.tagName}\n")

    for child in element.childNodes:
        if isinstance(child, Element):
            write_document_struct(child, level + 1)
```

```

def detect_task_type(content: str, task_type: str) -> dict:
    temp = ''

    if 'лабораторн' in content:
        temp = 'Лабораторная работа'
    elif 'курсов' in content:
        temp = 'Курсовая работа'
    elif 'контрольн' in content:
        temp = 'Контрольная работа'
    elif 'реферат' in content:
        temp = 'Реферат'

    if temp == task_type:
        return {"success": True, "data": temp}
    else:
        return {"success": False, "error": "Неверный тип задания"}

def detect_full_name_teacher(content: str, report_json: dict) -> dict:
    name_initial = report_json.get('teacher').get('name')[0] + '.'

    patronymic_initial =
report_json.get('teacher').get('patronymic', '')[0] + '.' \
        if report_json.get('teacher').get('patronymic') else ''

    surname = report_json.get('teacher').get('surname')

    teacher_full_name = name_initial + patronymic_initial + surname

    if teacher_full_name.lower() in content:
        return {"success": True, "data": teacher_full_name}
    else:
        return {"success": False, "error": "Неверное ФИО преподавателя"}

def detect_full_name_student(content: str, student_json: dict) -> dict:
    name_initial = student_json.get('name')[0] + '.'

    patronymic_initial = student_json.get('patronymic', '')[0] + '.' \
        if student_json.get('patronymic') else ''

    surname = student_json.get('surname')

    student_full_name = name_initial + patronymic_initial + surname

```

```

        if student_full_name.lower() in content:
            return {"success": True, "data": student_full_name}
        else:
            return {"success": False, "error": "Неверное ФИО студента"}

def detect_subject_name(content: str, subject_name: str) -> dict:
    if subject_name.replace(' ', '').lower() in content:
        return {"success": True, "data": subject_name}
    else:
        return {"success": False, "error": "Неверное название предмета"}

def detect_task_name(content: str, task_name: str) -> dict:
    if task_name.strip('JP1234567890.').replace(' ', '').lower() in content:
        return {"success": True, "data": task_name}
    else:
        return {"success": False, "error": "Неверное название работы"}

def detect_group(content: str, group: str) -> dict:
    if group.replace(' ', '').lower() in content:
        return {"success": True, "data": group}
    else:
        return {"success": False, "error": "Неверная группа"}

def detect_status(content: str, status: str) -> dict:
    if status.replace(' ', '').lower() in content:
        return {"success": True, "data": status}
    else:
        return {"success": False, "error": "Неверная должность преподавателя"}

def detect_report_structure(content: str, report_structure: list) -> dict:
    missing_points = []

    for point in report_structure:
        if point.replace(' ', '').lower() not in content:
            missing_points.append(point)

    if not missing_points:
        return {"success": True, "data": ', '.join(report_structure)}

```

```

        else:
            missing_points_str = ', '.join(missing_points)
            return {"success": False, "error": f"Отсутствуют пункты: {missing_points_str}"}

def detect_uploaded_at(content: str, date: str) -> dict:
    date = datetime.fromisoformat(date.replace("Z", "+00:00"))

    if str(date.year) in content:
        return {"success": True, "data": str(date.date())}
    else:
        return {"success": False, "error": 'Неверная дата'}

def get_keys_json(info_json: dict) -> list[str]:
    keys = []

    for key, value in info_json.items():
        keys.append(key)
        if isinstance(value, dict):
            keys.extend(get_keys_json(value))

    return keys

def detect_json(required_json: dict, input_json: dict) -> set[str]:
    required_keys = set(get_keys_json(required_json))
    input_keys = set(get_keys_json(input_json))

    required_keys.discard('patronymic')
    input_keys.discard('patronymic')

    return required_keys.difference(input_keys)

def check_report(report_odt: bytes, student_json: dict,
report_json: dict) -> list[str]:

    """
    Проверка документа OpenOffice .odt

    :param report_odt: File with a report in the form of a byte
stream

    :param student_json:
        { name: str,
          surname: str,
          patronymic: str | None,
          group: str
        }
    """

```

```

:param report_json:
    { subject_name: str,
      task_name: str,
      task_type: str,
      teacher: {
          name: str,
          surname: str,
          patronymic: str | None,
          status: str
      },
      report_structure: list[str],
      uploaded_at: str (ISO 8601 UTC)
    }

:return: List of errors
"""

missing_student_keys = detect_json(STUDENT_JSON,
student_json)
missing_report_keys = detect_json(REPORT_JSON, report_json)
# If the necessary keys are missing, the program shuts down
if missing_student_keys:
    return [f'Отсутствие обязательных ключей в student_json:
{"", ".join(missing_student_keys)}']
if missing_report_keys:
    return [f'Отсутствие обязательных ключей в report_json:
{"", ".join(missing_report_keys)}']

stream = io.BytesIO(report_odt)
document = opendocument.load(stream).body
remove_file_if_exists('../document_structure.txt')
write_document_struct(document) #
<office:body>...</office:body>
"""

    <office:body>
        <office:text>                                #
childNodes[0]
            <text:h>...</text:h>
            <text:p>...</text:p>
        </office:text>
    </office:body>
"""

content = [str(element) for element in
document.childNodes[0].childNodes if str(element) != '']
content = content[0].replace(' ', '').lower()

status = detect_status(content,
report_json.get("teacher").get("status"))
t_f_n = detect_full_name_teacher(content, report_json)
task_type = detect_task_type(content,
report_json.get("task_type"))

```



```

        task_name = detect_task_name(content,
report_json.get("task_name"))
        subject_name = detect_subject_name(content,
report_json.get("subject_name"))
        group = detect_group(content, student_json.get("group"))
        s_f_n = detect_full_name_student(content, student_json)
        report_structure = detect_report_structure(content,
report_json.get("report_structure"))
        uploaded_at = detect_uploaded_at(content,
report_json.get("uploaded_at"))

        print(f'Должность преподавателя: {status.get("data")}' if
status.get("success")
            else "\n\tERROR: " + status.get("error") + "\n")
        print(f'ФИО преподавателя: {t_f_n.get("data")}' if
t_f_n.get("success")
            else "\n\tERROR: " + t_f_n.get("error") + "\n")
        print(f'Тип отчета: {task_type.get("data")}' if
task_type.get("success")
            else "\n\tERROR: " + task_type.get("error") + "\n")
        print(f'Название работы: {task_name.get("data")}' if
task_name.get("success")
            else "\n\tERROR: " + task_name.get("error") + "\n")
        print(f'Название предмета: {subject_name.get("data")}' if
subject_name.get("success")
            else "\n\tERROR: " + subject_name.get("error") + "\n")
        print(f'Номер группы: {group.get("data")}' if
group.get("success")
            else "\n\tERROR: " + group.get("error") + "\n")
        print(f'ФИО студента: {s_f_n.get("data")}' if
s_f_n.get("success")
            else "\n\tERROR: " + s_f_n.get("error") + "\n")
        print(f'Структура отчета: {report_structure.get("data")}' if
report_structure.get("success")
            else "\n\tERROR: " + report_structure.get("error") +
"\n")
        print(f'Дата загрузки: {uploaded_at.get("data")}' if
uploaded_at.get("success")
            else "\n\tERROR: " + uploaded_at.get("error") + "\n")

        response_data = [status, t_f_n, task_type, task_name,
subject_name, group, s_f_n, report_structure, uploaded_at]
        errors = [elem.get("error") for elem in response_data if not
elem.get("success")]

        return errors

def main():
    cur_dir = os.path.dirname(__file__)
    report_odt_def_path = os.path.join(cur_dir, '..', 'data',
'correct_lab.odt')

```

```

    student_json_def_path = os.path.join(cur_dir, '..', 'data',
'student_info.json')
    report_json_def_path = os.path.join(cur_dir, '..', 'data',
'report_info.json')

    report_odt_def_path = os.path.abspath(report_odt_def_path)
    student_json_def_path =
os.path.abspath(student_json_def_path)
    report_json_def_path = os.path.abspath(report_json_def_path)

    parser = argparse.ArgumentParser(description='Модуль
проверки отчетов в формате OpenOffice')
    parser.add_argument('--report_odt',
default=report_odt_def_path, help='Путь к файлу отчета .odt')
    parser.add_argument('--student_json',
default=student_json_def_path, help='Путь к файлу с информацией
о студенте .json')
    parser.add_argument('--report_json',
default=report_json_def_path, help='Путь к файлу с информацией
об отчете .json')

    args = parser.parse_args()

    with open(file=args.report_odt, mode='rb') as
report_data_odt:
        report_odt = report_data_odt.read()

    with open(file=args.student_json, mode='r', encoding='utf-
8') as student_data_json:
        student_json = json.load(student_data_json)

    with open(file=args.report_json, mode='r', encoding='utf-8')
as report_data_json:
        report_json = json.load(report_data_json)

    errors = check_report(report_odt, student_json, report_json)

if __name__ == '__main__':
    main()

```

REPORT_INFO.JSON

```
{
  "subject_name": "Операционные системы",
  "task_name": "ЛР1. Знакомство с командным интерпретатором
bash",
  "task_type": "Лабораторная работа",
  "teacher": {
    "name": "Марк",
    "surname": "Поляк",
    "patronymic": "Дмитриевич",
    "status": "Старший преподаватель"
  },
  "report_structure": [
    "Цель", "Задание", "Результат выполнения", "Выводы"
  ],
  "uploaded_at": "2024-06-01T00:00:00Z"
}
```

STUDENT_INFO.JSON

```
{
  "name": "Владимир",
  "surname": "Иванов",
  "patronymic": "Петрович",
  "group": "4133К"
}
```

TEST_CHECK_REPORT_MODULE.PY

```
import os
import copy

from check_report.check_report_module import check_report

student_json = { "name":
    "Владимир",
    "surname": "Иванов",
    "patronymic": "Петрович",
    "group": "4133К"
}

report_json = {
    "subject_name": "Операционные системы",
    "task_name": "ЛР1. Знакомство с командным интерпретатором
bash",
    "task_type": "Лабораторная работа",
    "teacher": {
        "name": "Марк",
        "surname": "Поляк", "patronymic":
        "Дмитриевич", "status": "Старший
преподаватель"
    },
    "report_structure": [
        "Цель", "Задание", "Результат выполнения", "Выводы"
    ],
    "uploaded_at": "2024-06-01T00:00:00Z"
}

cur_dir = os.path.dirname(__file__)
report_odt_def_path = os.path.join(cur_dir, '..', 'data',
'correct_lab.odt')
report_odt_def_path = os.path.abspath(report_odt_def_path)

with open(file=report_odt_def_path, mode='rb') as
report_data_odt:
    report_odt = report_data_odt.read()

def test_correct_report():
    errors = check_report(report_odt, student_json, report_json)
    assert len(errors) == 0

def test_missing_student_keys():
    student_keys_error = student_json.copy()
```

```

        student_keys_error.pop('surname', None)
        errors = check_report(report_odt, student_keys_error,
report_json)
        assert "Отсутствие обязательных ключей в student_json:
surname" in errors and len(errors) == 1

def test_missing_report_keys():
    report_keys_error = copy.deepcopy(report_json)
    report_keys_error['teacher'].pop('surname', None)
    errors = check_report(report_odt, student_json,
report_keys_error)
    assert "Отсутствие обязательных ключей в report_json:
surname" in errors and len(errors) == 1

def test_incorrect_student_group():
    student_group_error = student_json.copy()
    student_group_error['group'] = '1111K'
    errors = check_report(report_odt, student_group_error,
report_json)
    assert "Неверная группа" in errors and len(errors) == 1

def test_incorrect_teacher_status():
    teacher_status_error = copy.deepcopy(report_json)
    teacher_status_error['teacher']['status'] = 'Директор'
    errors = check_report(report_odt, student_json,
teacher_status_error)
    assert "Неверная должность преподавателя" in errors and
len(errors) == 1

def test_incorrect_report_structure():
    report_structure_error = copy.deepcopy(report_json)
    report_structure_error['report_structure'] =
report_structure_error['report_structure']
    report_structure_error['report_structure'].append('Список
источников')
    errors = check_report(report_odt, student_json,
report_structure_error)
    assert "Отсутствуют пункты: Список источников" in errors and
len(errors) == 1

def test_incorrect_uploaded_at():
    report_uploaded_at_error = copy.deepcopy(report_json)
    report_uploaded_at_error['uploaded_at'] = '2023-06-
01T00:00:00Z'
    errors = check_report(report_odt, student_json,
report_uploaded_at_error)
    assert "Неверная дата" in errors and len(errors) == 1

```

```

def test_incorrect_subject_name():
    report_subject_name_error = copy.deepcopy(report_json)
    report_subject_name_error['subject_name'] = 'Архитектура ЭВМ'
    errors = check_report(report_odt, student_json, report_subject_name_error)
    assert "Неверное название предмета" in errors and len(errors) == 1

def test_incorrect_task_name():
    report_task_name_error = copy.deepcopy(report_json)
    report_task_name_error['task_name'] = 'ЛР2. Разработка многопоточного приложения ' \
                                           'средствами POSIX в ОС Linux или Mac OS'
    errors = check_report(report_odt, student_json, report_task_name_error)
    assert "Неверное название работы" in errors and len(errors) == 1

def test_incorrect_task_type():
    report_task_type_error = copy.deepcopy(report_json)
    report_task_type_error['task_type'] = 'Контрольная работа'
    errors = check_report(report_odt, student_json, report_task_type_error)
    assert "Неверный тип задания" in errors and len(errors) == 1

def test_incorrect_student_name():
    student_name_error = student_json.copy()
    student_name_error['name'] = 'Иван'
    errors = check_report(report_odt, student_name_error, report_json)
    assert "Неверное ФИО студента" in errors and len(errors) == 1

def test_incorrect_student_surname():
    student_surname_error = student_json.copy()
    student_surname_error['surname'] = 'Петров'
    errors = check_report(report_odt, student_surname_error, report_json)
    assert "Неверное ФИО студента" in errors and len(errors) == 1

def test_incorrect_student_patronymic():
    student_patronymic_error = student_json.copy()

```

```

    student_patronymic_error['patronymic'] = 'Иванович'
    errors = check_report(report_odt, student_patronymic_error,
report_json)
    assert "Неверное ФИО студента" in errors and len(errors) ==
1

def test_incorrect_teacher_name():
    teacher_name_error = copy.deepcopy(report_json)
    teacher_name_error['teacher']['name'] = 'Иван'
    errors = check_report(report_odt, student_json,
teacher_name_error)
    assert "Неверное ФИО преподавателя" in errors and
len(errors) == 1

def test_incorrect_teacher_surname():
    teacher_surname_error = copy.deepcopy(report_json)
    teacher_surname_error['teacher']['surname'] = 'Петров'
    errors = check_report(report_odt, student_json,
teacher_surname_error)
    assert "Неверное ФИО преподавателя" in errors and
len(errors) == 1

def test_incorrect_teacher_patronymic():
    teacher_patronymic_error = copy.deepcopy(report_json)
    teacher_patronymic_error['teacher']['patronymic'] =
'Иванович'
    errors = check_report(report_odt, student_json,
teacher_patronymic_error)
    assert "Неверное ФИО преподавателя" in errors and
len(errors) == 1

def test_multiple_errors():
    student_errors = student_json.copy()
    report_errors = copy.deepcopy(report_json)
    report_errors['subject_name'] = 'Архитектура ЭВМ'
    student_errors['group'] = '1111К'
    report_errors['uploaded_at'] = '2023-06-01T00:00:00Z'
    expected_errors = {'Неверное название предмета', 'Неверная
группа', 'Неверная дата'}
    errors = check_report(report_odt, student_errors,
report_errors)
    assert expected_errors == set(errors) and len(errors) == 3

```